Embedded Real-Time Control, Alberto Morato

Laboratory 2, Motor Parameters Identification

Last update: 25-03-2024 Hash: 47b0b55-dirty

1 Introduction

Purpose of this laboratory is to identify the first-order model of the DC motors mounted on the TBot. The identification will be performed by controlling in open-loop the motors, by applying a known step voltage input and measuring the angular speed response. The collected data will be used to estimate the motor parameters.

2 Preliminary Information

2.1 Data logger

The TBot is equipped with a data logger that can record almost any type of data. There are two available communication interfaces:

- Serial Datalogger (via USB debug interface)
- WiFi Datalogger

2.1.1 MATLAB

The serial datalogger can be invoked from MATLAB using the command:

```
data = serial_datalog(COM_port , packet_specification)
```

where `COM_port` is the port of the serial datalogger (e.g., COM3) and `packet_specification` is a cell array of char arrays that specifies the type of data to be recorded. The port currently in use can be immediately detected by invoking the `seriallist` routine from the MATLAB Command, provided that the STM32F7 board is the only device connected to the host computer. If so, the routine should return a list of two items, of which the first is the COM1 port reserved by the system for internal uses, and the other is the port used by the STM32F7 board to communicate with the host computer.

The data received from the robot are simultaneously displayed on a MATLAB figure in real-time and saved into a memory buffer. Once the figure is closed, the buffer content is copied into the output variable `data`, which is a MATLAB structure with the following fields:

- `data.time` is the field containing the time instants at which the data samples have been transmitted, with respect to the beginning of the data logging process.
- `data.out` is a cell array containing the data samples sent by the balancing robot. Each element of the cell array corresponds to a different logged signal. It is a matrix, whose columns are the values of the logged signal. Therefore, the number of rows is equal to the size of the signal (number of components), and the number of columns is instead equal to the number of received data samples. The number of received data samples never exceeds the size of the buffer used to hold the data during the data logging process (by default, the buffer size is equal to 1000). For example, the command `y = data.out{2}(2,:)` retrieves the 2nd component of the 2nd signal sent by the STM32F7 board.

For example, by invoking the command:

```
data = serial_datalog('COM8',{'2*single','2*single'}, 'baudrate', 115200, 'buffersize', 100000)
```

the data logger will expect to receive a total of four single precision floating point numbers, which will be displayed in two separate sub-plots. The baudrate is 115200. Using the `buffersize` argument, is possible to change how many sample to save.

To use the WiFi datalogger, the following command should be invoked:

```
data = udp_datalog(ip, port, packet_specification)
```

where `ip` is the IP address of the robot, `port` is the port number used by the WiFi datalogger, and `packet_specification` is a cell array of char arrays that specifies the type of data to be recorded.

For example, you can invoke the command:

```
data = udp_datalog('147.162.118.61',9090, {'2*single','2*single'}, 'buffersize', 100000)
```



ATTENTION: every turtlebot has a different IP address. Please check (or ask for it) the IP address of the robot you are trying to connect to.

If something hangs or crashes during the data logging process and nothing is working anymore, you can try those two commands:

```
clear all
instrreset
```

2.1.2 STM32

On the STM32 side, the data logger must be properly configured. Example code is provided in the project for Lab 2.

You can choose which communication interface you want to use by calling the following instructions inside the main function (these function calls are already included in the project):

```
//logger.uart_handle = huart3; // for serial
logger.uart_handle = huart2; // for wifi
```

The basic steps to initialize and run the data logger are explained in the code below:

```
/* declare an ertc_dlog struct */
struct ertc_dlog logger;
.
.
.
int main(void)
{
    .
    .
    .
    /* USER CODE BEGIN 2 */
    /* select the correct UART: uart3 is the default peripheral used */
    logger.uart_handle = huart3;
    // logger.uart_handle = huart2; // for wifi

    /* in this example, we use the timer to periodically send data through the data logger */
    HAL_TIM_Base_Start_IT(&htim6);
    /* USER CODE END 2 */
    .
    .
    .
    while (1)
    {
        /* USER CODE BEGIN 3 */
        /* continuously update the state of the data logger */
        ertc_dlog_update(&logger);
    }
    /* USER CODE END 3 */
}
```

To send arbitrary data, it is necessary to define a struct as follows:

```
struct datalog
{
    float w1, w2;
    float u1, u2;
} data;
```

The structure above is just an example; the structure can be arbitrary, but it is important to put the `datalog` attribute after the word `struct`.

To send the data:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM6)
    {
        data.w1 = 10;
```

```
6           data.w2 += 1.085;
7           data.u1 = -3.14;
8           data.u2 = 0.555683;
9           ertc_dlog_send(&logger, &data, sizeof(data));
10      }
11  }
```

To avoid the case in which the board starts sending "useful" data before MATLAB is ready to show them, it is suggested to check for the value of `tx_enabled` before setting the value of the reference signal. `tx_enabled` indicates whenever the data-logger, from the PC side, is able to receive data.

 **WARNING**: To avoid excessive transmission time and delays please send only numeric data (e.g., float, int, uint8_t, etc.). Do **not** send strings.

3 Exercises

3.1 Exercise 1

- The main goal for this laboratory is to identify the first-order model of the DC motors mounted on the TBot.
- This is done by applying a step voltage input to the motors and measuring the angular speed response.
- According to the **Motor Control lecture**, you need to implement:
  - All the necessary functions to read and compute the angular speed of the motors. (Examples are provided in the Motor Control lecture document.)
  - All the necessary functions to set the voltage applied to the motors using PWM. (Examples are provided in the Motor Control lecture document.)
  - All the steps needed to accomplish the goal should be executed from inside the `HAL_TIM_PeriodElapsedCallback` function, which, as previously mentioned, is a callback function called when a timer's period is elapsed. Remember to check which timer caused the callback to be called: in particular, we are interested in handling the events raised by the timer TIM6.
- According to the **Motor Parameters identification lecture**, acquire the step response of **both the motors** by applying a step voltage input and measuring the angular speed response.
  - Selection of the step amplitude is left to the student. The choice must be motivated. Is suggested to acquire multiple step responses with different amplitudes.
  - Selection of the control mode (e.g., Forward/Brake or Forward/Coast) is left to the student. You can select one or try both. The choice must be motivated.
  -  **WARNING**: The maximum voltage for the motor is 6V. Supposing a 12V power supply, the maximum duty cycle for the PWM signal **needs to be limited to 50%**.
- With the collected data, estimate the motor parameters using the **tangent method** and one of the other two methods presented in the Motor Parameters identification lecture.
  - Compare the results obtained with the different methods.
- According to the identified first-order model, find the values of the k_p and k_i constants for a PI controller that regulates the angular speed of the motors.
  - The technique to find the k_p and k_i constants is left to the student. You can use any method presented in the lectures or any other method you prefer. The choice must be motivated.
- Analyze in Matlab the closed-loop step response of the system with the designed PI controller.
- **The found k_p and k_i constants will be used in next laboratory sessions to implement the PI controller on the TBot.**